React Basics

React Components, JSX, TSX

Svetlin Nakov, PhD

Co-founder @ SoftUni

SoftUni AI

https://ai.softuni.bg

Agenda

SoftUni AI

1. **Introduction to React and JSX**: component-based architecture, React components, JSX syntax, UI manipulation, React-DOM libraries
2. **Initializing a React Project with TypeScript**: npm create vite@latest
3. **React with TypeScript**: project structure, folders, files, configs, CSS styles
4. **Simple React Components**: TSX, rendering, passing parameters, styling
5. **React Components with State**: React hooks, useState(), handling events
6. **React Dev Tools**: Installing Dev Tools, inspecting app components
7. **Nested Components**: defining components, CSS styling
8. **Rendering Lists**: rendering multiple items, conditional rendering
9. **Multi-Select Component**: creating reusable components

Sli.do Code

AppsAI

Join at

slido.com

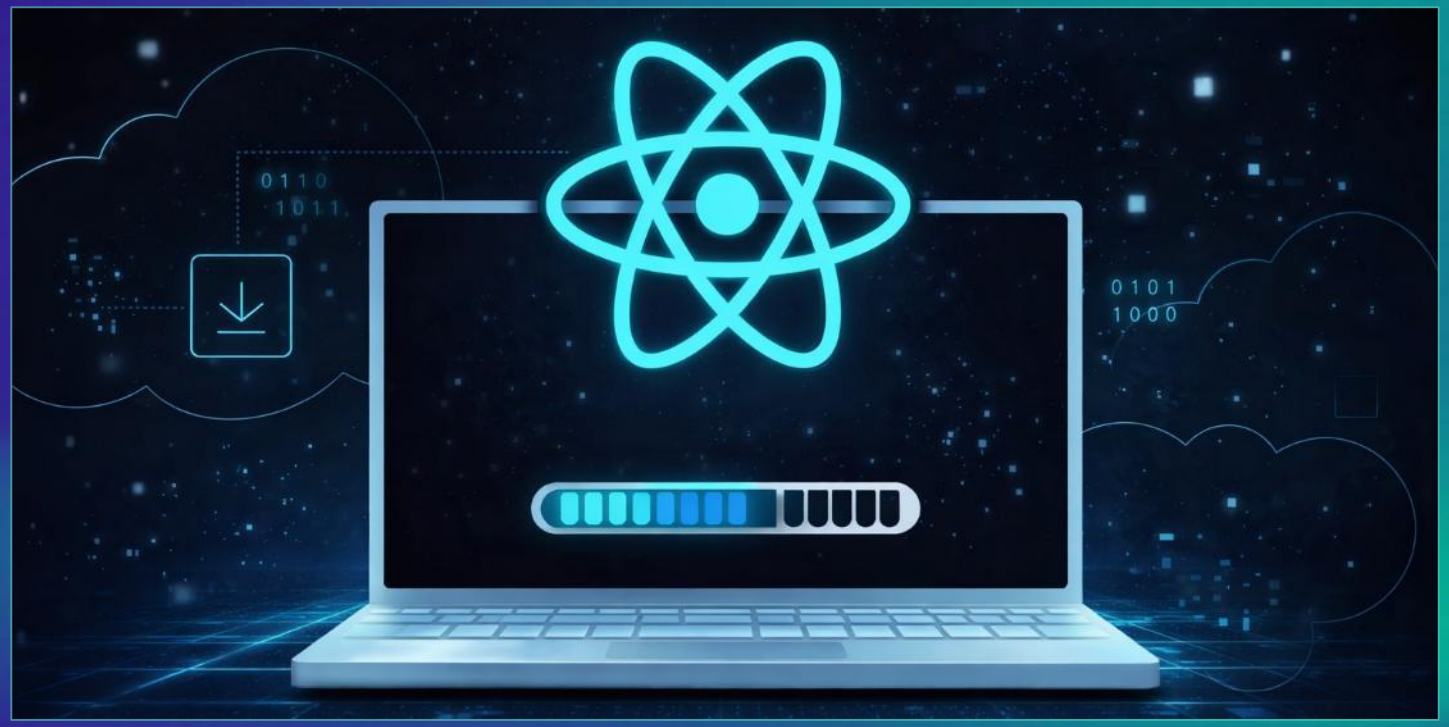
#AppsAI

Breaks

20:00 / 21:00

Introduction to React and JSX

Component-Based Architecture, React Components, JSX Syntax, React DOM



What is React?

SoftUni AI

- **React** is a **JavaScript library** for building user interfaces
  - Developed and maintained by **Meta** (Facebook)
  - Released in 2013, **open-source** (MIT License)
  - Extremely **popular**: 20M downloads / week, 70M sites
  - https://react.dev

React

- Key features:
  - **Component-based architecture** – build UIs from reusable pieces
  - **JSX syntax** – mix HTML, CSS and JavaScript into a single JS file
  - **Virtual DOM** – efficiently updates only what changes

React – Example

```
export default function Hello() {
  const name = "SoftUni";
  const styles = { color: "blue", fontSize: "20px" };
  return (
    <div>
      <h1 style={styles}>Hello, {name}!</h1>
      <button onClick={() => alert("Clicked!")}>
        Click me
      </button>
    </div>
  );
}
```

React component function

Mix HTML with JS code

Component-Based Architecture

SoftUni AI

- In React the UI is built from reusable pieces called **components**
  - **Composable** (nested and combined)
- A **React component** is a **JS function** (or class) that contains:
  - **JSX** (JS with HTML-like structure)
  - **JavaScript logic** (rendering, events, …)
  - Optional **styles**
  - **State** (data that changes over time)

```
<Layout>
  <Header />
  <div>…</div>
  <Footer />
</Layout>
```

- **Benefits**:
  - Reusable
  - Testable
  - Maintainable

React Features

SoftUni AI

- React ecosystem covers many layers of modern front-end:

UI Components
Component model with JSX/TSX syntax

Hooks
useState(), useEffect(), useContext()

State Management
useState(), useReducer(), Redux, etc.

Routing
React Router / Next.js router

HTTP Requests
useEffect(), fetch, Axios for data fetching

Memoization
useMemo(), useCallback()

Styling
CSS styles, styled-components, Tailwind

Form Validation
React Hook Form, Zod

JSX Syntax

SoftUni AI

- **JSX** == JavaScript with XML
- Insert **HTML fragments** directly into the JS code
- Return a **single root** element, or use a fragment **<>…</>**
- HTML element must be closed: **
, **<img />
- Use "**className**" instead of "**class**" for styles
- Insert other React components as HTML tags, e.g. **<Footer />**

```
import Footer from "./Footer";

export function Greeting() {
  const name = "React";
  return (
    <>
      <div className="card">
        <h1>Hey {name},</h1>
        <p>Welcome here!</p>
      </div>
      <Footer />
    </>
  );
}
```

JSX vs. TSX Components

- **JSX** == React component with JavaScript
  - **Component.jsx**
  - No type checking

- **TSX** == React component with TypeScript
  - **Component.tsx**
  - Static type checking
  - Compile-time errors

```
export function Button({ label }) {
  return <button>{label}</button>;
}
```

```
interface ButtonProps {
  label: string;
  onClick?: () => void;
}
export function Button(
{ label, onClick }: ButtonProps) {
  return (
    <button onClick={onClick}>
{label}</button>
  );
}
```

Approaches to UI Manipulation

SoftUni AI

- **Declarative** (React) vs. **imperative** (Vanilla JS) approach to UI manipulation

- **Declarative** UI rendering:

```
function Counter({ count }) {
  return <h1>{count}</h1>;
}
```

  - Easy to write / maintain
  - UI is rendered by the framework, based on the component state

- **Imperative** UI rendering:

```
const h1 =
  document.querySelector("h1");
h1.textContent = count;
```

  - Manually update UI
  - No clear concept of component and state
  - More code, more complexity

Imperative vs. Declarative UI

SoftUni AI

✨ Declarative UI — React

- Describe **WHAT** the UI should look like (React handles it)

```
function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <p>{count}</p>
      <button onClick={() =>
        setCount(count + 1)}>+1</button>
    </div>
  );
}
```

⚙️ Imperative UI — Vanilla JS

- Tell the browser **HOW** to update the DOM (step by step instructions)

```
<div id="count">0</div>
<button id="btnIncr">+1</button>
```

```
const btn =
  document.getElementById("btnIncr");
btn.addEventListener("click", () => {
  const counter =
    document.getElementById("count");
  counter.textContent =
    Number(counter.textContent) + 1;
});
```

React and React DOM Libraries

SoftUni AI

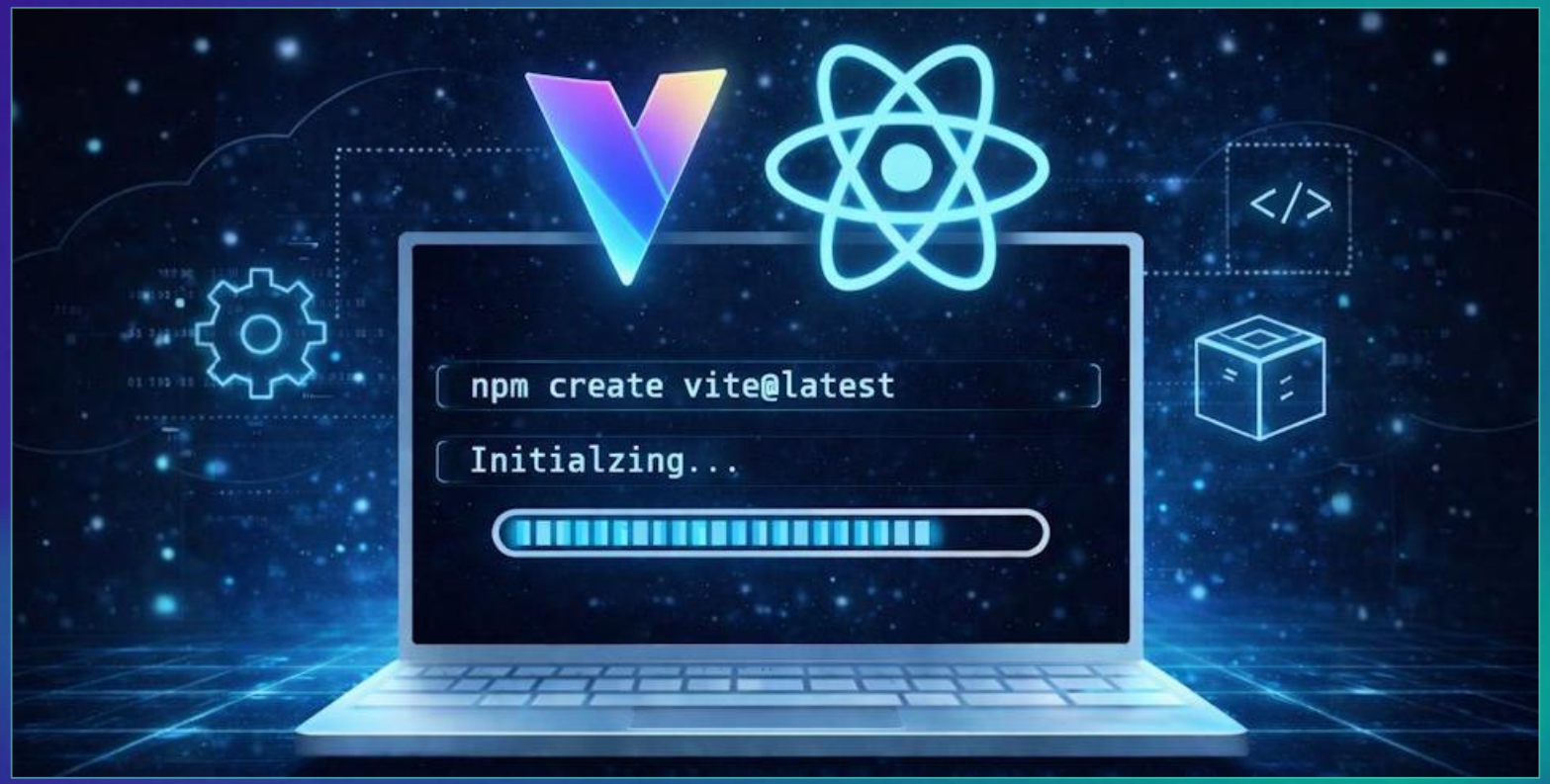
- Library "**react**": components, hooks, JSX transformation
- Library "**react-dom**":
  - Renders React in **Web** browsers (handles all DOM manipulations)
- Library "**react-native**":
  - Renderer for **iOS** and **Android** mobile apps (uses native components)

```
// main.tsx — application entry point
import { StrictMode } from "react";
import { createRoot } from
  "react-dom/client";
```

```
import App from "./App.tsx";
import "./App.css";

createRoot(document.getElementById(
"root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

Initializing a React Project

npm create vite@latest



Creating a React Project

SoftUni AI

- Initializing a **React with TypeScript project**:

```
npm create vite@latest
```

  - Select a framework: **React**
  - Select a language: **TypeScript**

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
PS React-Basics> npm create vite@latest
|
◆ Select a framework:
  ○ Vanilla
  ○ Vue
  ● React
  ○ Preact
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
○ PS React-Basics> npm create vite@latest

  > react-basics@0.0.0 npx
  > create-vite

  |
  ◆ Project name:
  | react-app
  └
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS React-Basics> npm create vite@latest
|
◆ Select a variant:
  ● TypeScript
  ○ TypeScript + React Compiler
  ○ JavaScript
```

Creating a React Project (2)

SoftUni AI

- **Installing** project dependencies:

```
npm install
```

- **Running** the project in dev mode:

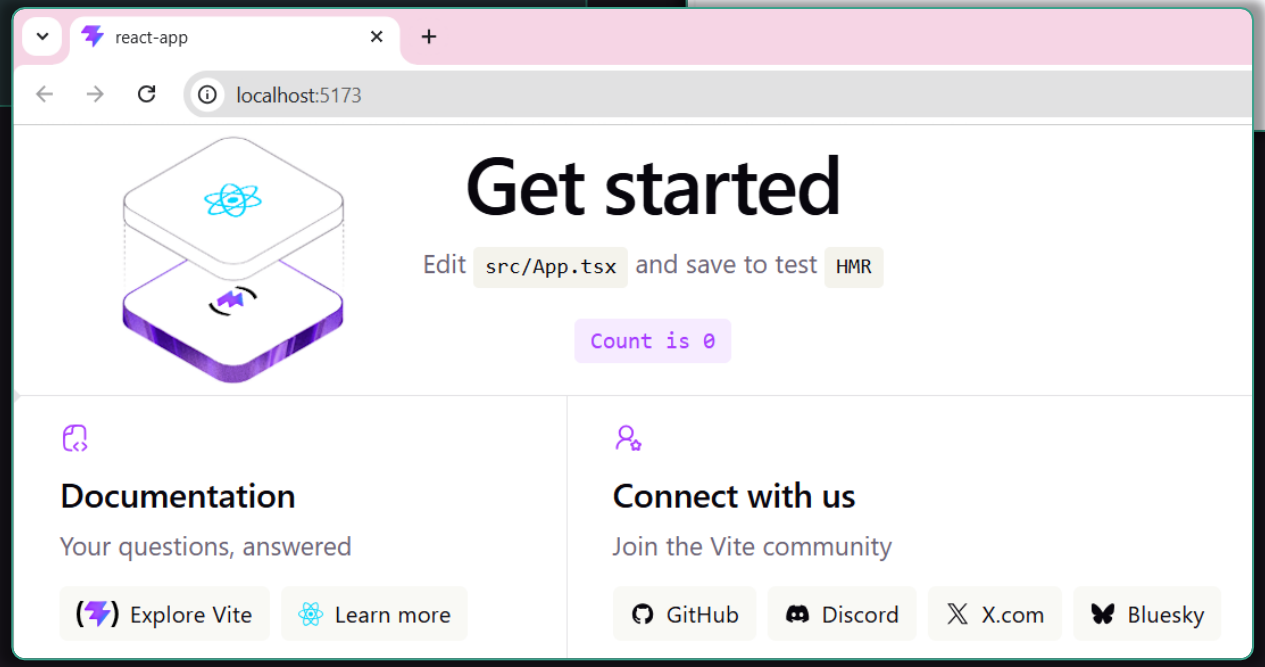
```
npm run dev
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
PS React-Basics> npm install

up to date, audited 67 packages in 914ms

12 packages are looking for funding
  run `npm fund` for details
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

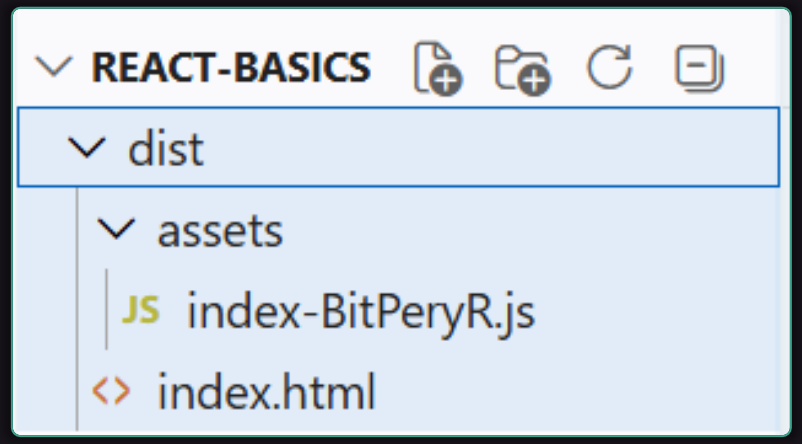
```
PS React-Basics> npm run dev

> react-basics@0.0.0 dev
> vite


  VITE v8.0.3  ready in 212 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help
```



Vite + React: Key Concepts

SoftUni AI

- **Vite** is fast **build tool** for JS devs, with full React support
- **HMR** (Hot Module Replacement)
  - Updates the browser instantly on save (on code changes)
  - No full page reload needed during development
- **Production build** `npm run build`
  - Create an optimized production bundle in the `**dist**` folder
- **Preview** `npm run preview`
  - Preview the optimized production build in the browser



React with TypeScript

Project Structure, Folders, Files, Configs

Vite + React: Folders, Files, Configs

```
react-app/                  # Project root folder
├── public/                 # Public site assets: static files, icons, fonts (stay untouched)
│   └── favicon.svg
├── src/                    # App source code (React components, scripts, styles)
│   ├── assets/             # Vite app assets: images, icons (processed & optimized by Vite)
│   │   └── react.svg
│   ├── App.tsx             # App main component (rendered by main.tsx)
│   ├── App.css             # App component CSS styles (imported from App.tsx)
│   ├── index.css           # App's global CSS (imported from main.tsx)
│   └── main.tsx            # App starting point → renders the <App /> component
├── index.html              # App entry point: loads React-DOM and renders main.tsx
├── package.json            # Node.js project config: package dependencies, startup scripts
├── tsconfig.json           # Config settings for the TypeScript compiler
└── vite.config.ts          # Config settings for the Vite environment
```

Vire + React: Configuration Files

SoftUni AI

package.json
Dependencies, scripts, and project metadata

vite.config.ts
Vite build tool config: plugins, aliases, etc.

eslint.config.js
ESLint settings: checks the code for problems

tsconfig.json
Project's TypeScript compiler configuration

package.json

```
{
  "scripts": {
    "dev": "vite",
    "build": "tsc -b && vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^19.2.4",
    "react-dom": "^19.2.4"
  },
}
```

index.html → main.tsx → App.tsx

index.html

```html
<body>
  <div id="root"></div>
  <script type="module"
    src="/src/main.tsx"></script>
</body>
```

main.tsx

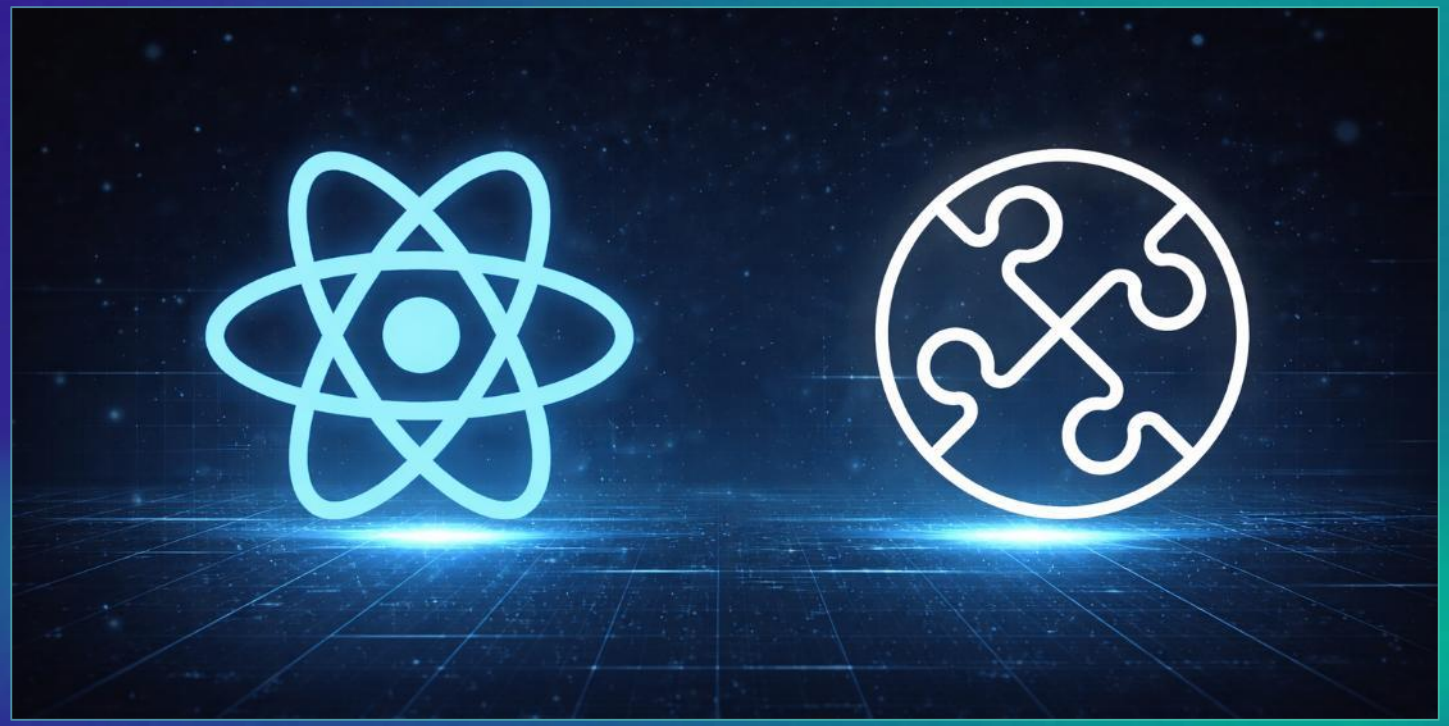
```tsx
import App from './App.tsx';
createRoot(document.getElementById(
'root')!).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

App.tsx

```tsx
export default function App() {
  return (
    <h1>Hello from App.tsx</h1>
  )
}
```

Simple React Components

TSX Components, UI Rendering,
Passing Parameters, Styling with CSS



Creating Simple React TSX Component

SoftUni AI

- AI prompt to **create a React component**:

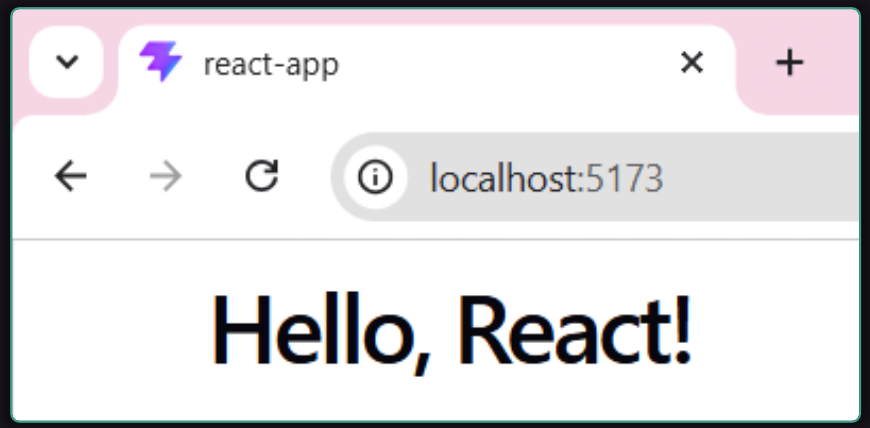
Create a simple **React TSX component**:
- function **Greeting()** -> Hello, React!
- Render the component: <Greeting />

in App.tsx

App.tsx

react-app > src > App.tsx > default

```
1  function Greeting() {
2    return <h1>Hello, React!</h1>
3  }
4
5  function App() {
6    return <Greeting />
7  }
8
9  export default App;
```



Passing Parameters to Components

SoftUni AI

- AI prompt to create a **React component with parameters**:

Create a component: `**function Greeting(props) -> Hello, {props.name}!**`
- Render the component: <Greeting name="SoftUni" />

App.tsx

react-app > src > App.tsx > default

```
1  type GreetingProps = {
2    name: string
3  }
4
5  function Greeting(props: GreetingProps) {
6    return <h1>Hello, {props.name}!</h1>
7  }
8
9  function App() {
10   return <Greeting name="SoftUni" />
11 }
```

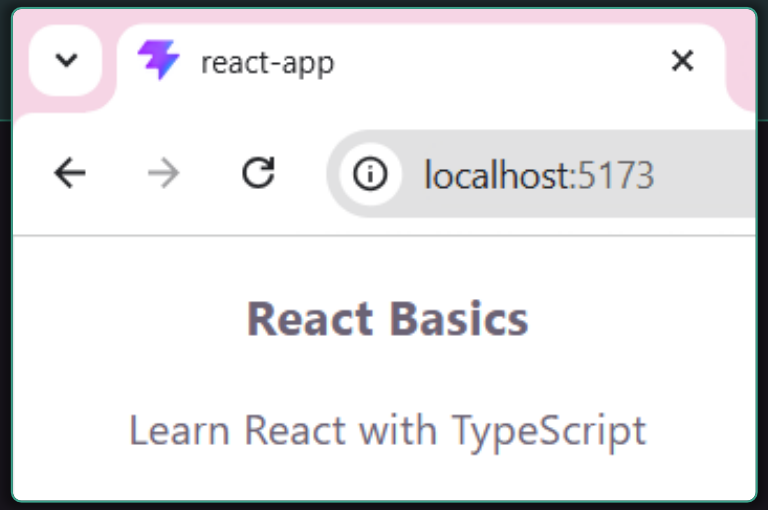
Change component's parameters:
- `function Greeting({**firstName**, **lastName**}) -> Hello, {**firstName**} {**lastName**}!`
- Render the component: <Greeting **firstName**="Steve" **lastName**="Smith" />

Components with Props – Example

```
interface CardProps {
  title: string;
  description: string;
  imageUrl?: string;
  onClick: () => void;
}
```

```
function Card({ title, description, imgUrl, onClick }:
CardProps) {
  return (
    <div className="card" onClick={onClick}>
      {imageUrl && <img src={imgUrl} alt={title} />}
      <h3>{title}</h3>
      <p>{description}</p>
    </div>
  )}
```

```
<Card
  title="React Basics"
  description="Learn React with TypeScript"
  onClick={() => console.log("clicked")}
/>
```



Styling Components with CSS

- **Styling** a React component with **CSS**:

index.css
Global CSS styles for the entire app

Item.css
Per component styles (not scoped)

Area.module.css
CSS module: no class name conflicts

Item.css

```
.container { background: #eee }
.text { color: darkblue }
```

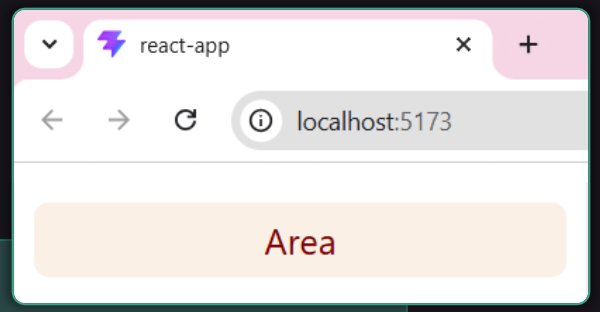
Item.tsx

```
import './Item.css'

function Item() {
  return (
    <div className="container">
      <p className="text">Item</p>
    </div>
)}
```

The class name is global: possible name conflicts

Styling with Module CSS

- Styling with **module CSS** solves name conflicts



Area.tsx

```
import styles from "./Area.module.css";

export function Area() {
  return (
    <div className={styles.container}>
      <p className={styles.text}>Area</p>
    </div>
  );
}
```

Vite will rewrite the class name to `_container_7ac1`

Area.module.css

```
.container {
  background: linen;
  border-radius: 8px;
  padding: 8px;
  margin: 10px;
}

.text {
  color: maroon;
  font-size: 18px;
}
```

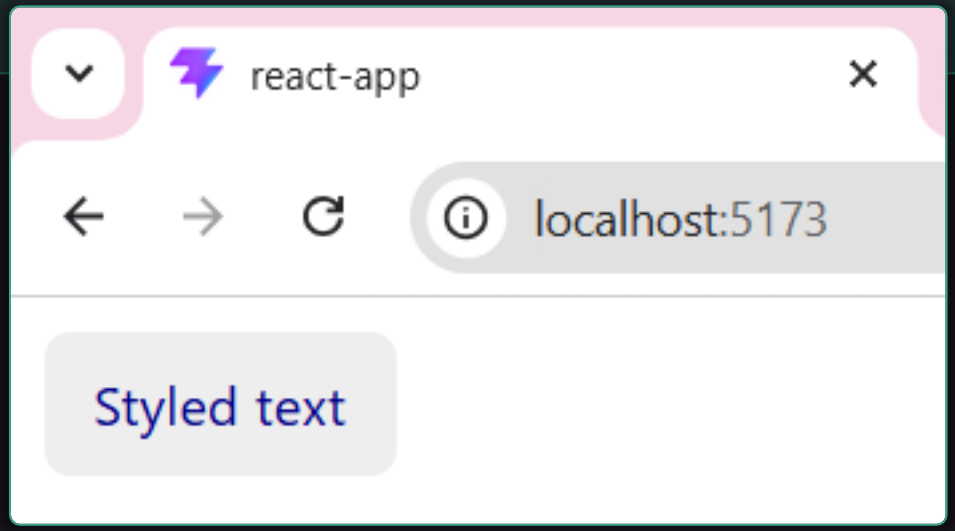
Styling Components with JS Styles

SoftUni AI

- **Styling** a component with a **JS style object**

```
const styles = {
  container: {
    backgroundColor: "#eee",
    borderRadius: "8px",
    display: "inline-block",
    margin: "10px",
    padding: "10px 15px"
  },
  text: { color: "darkblue" }
};
```

```
function Box() {
  return (
    <div style={styles.container}>
      <p style={styles.text}>Text</p>
    </div>
  )
}
```

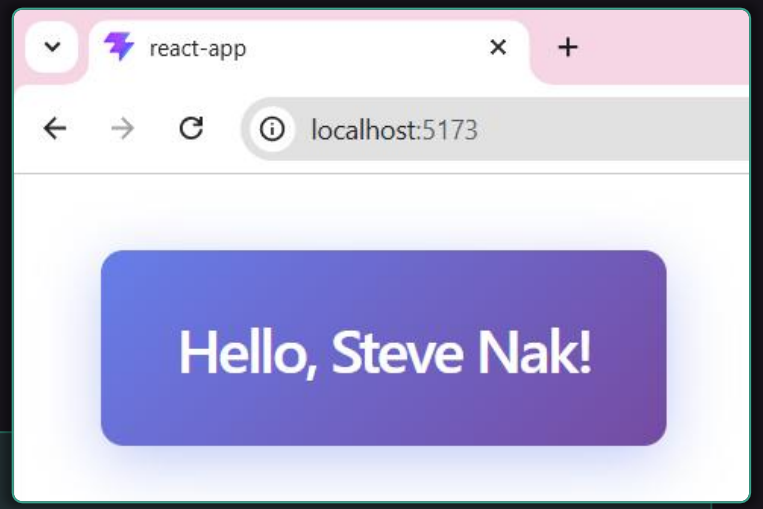


Styling a Component – Example

SoftUni AI

- AI prompt to **style a React component**:

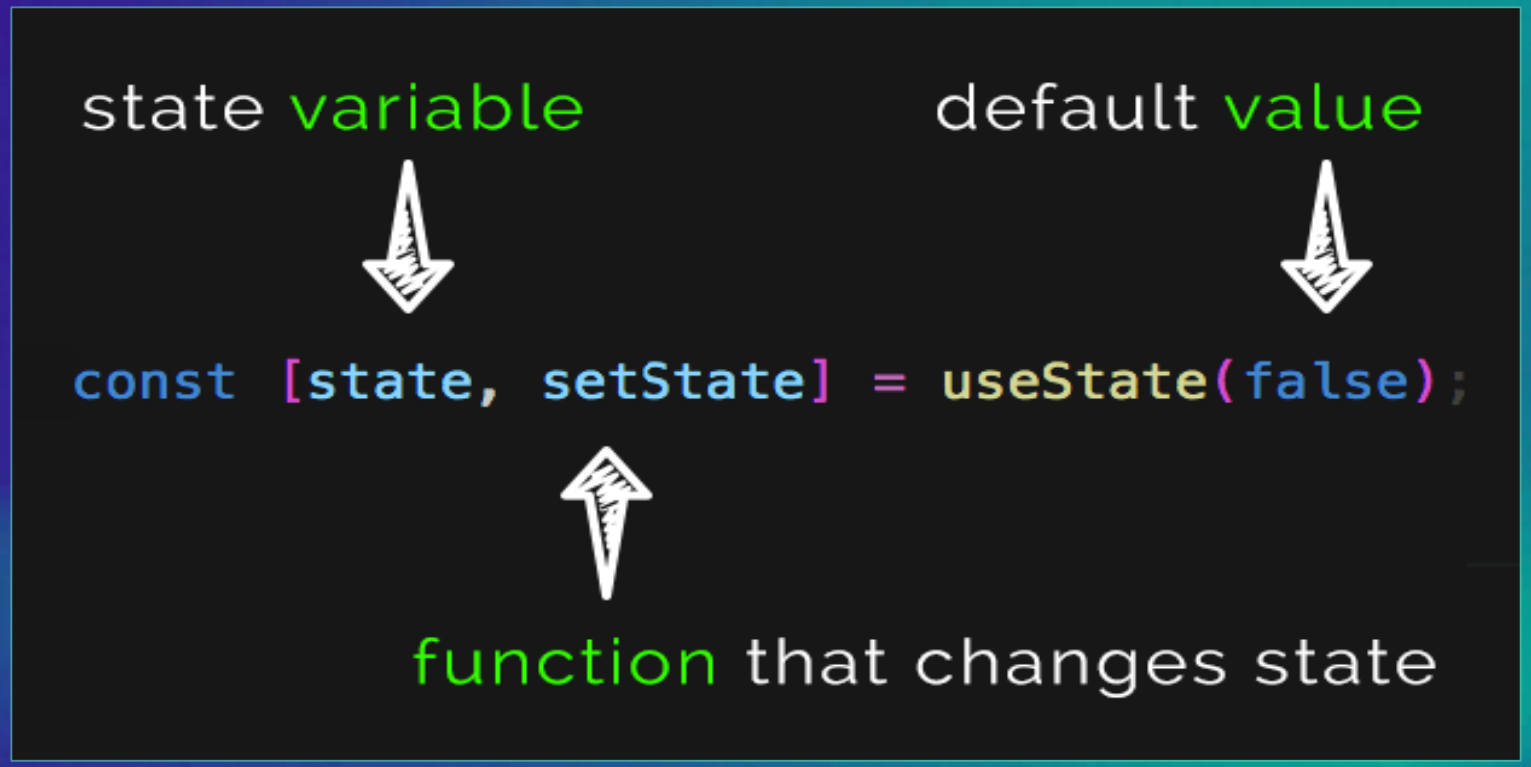
Add CSS to visualize the <Greeting />
component as a card



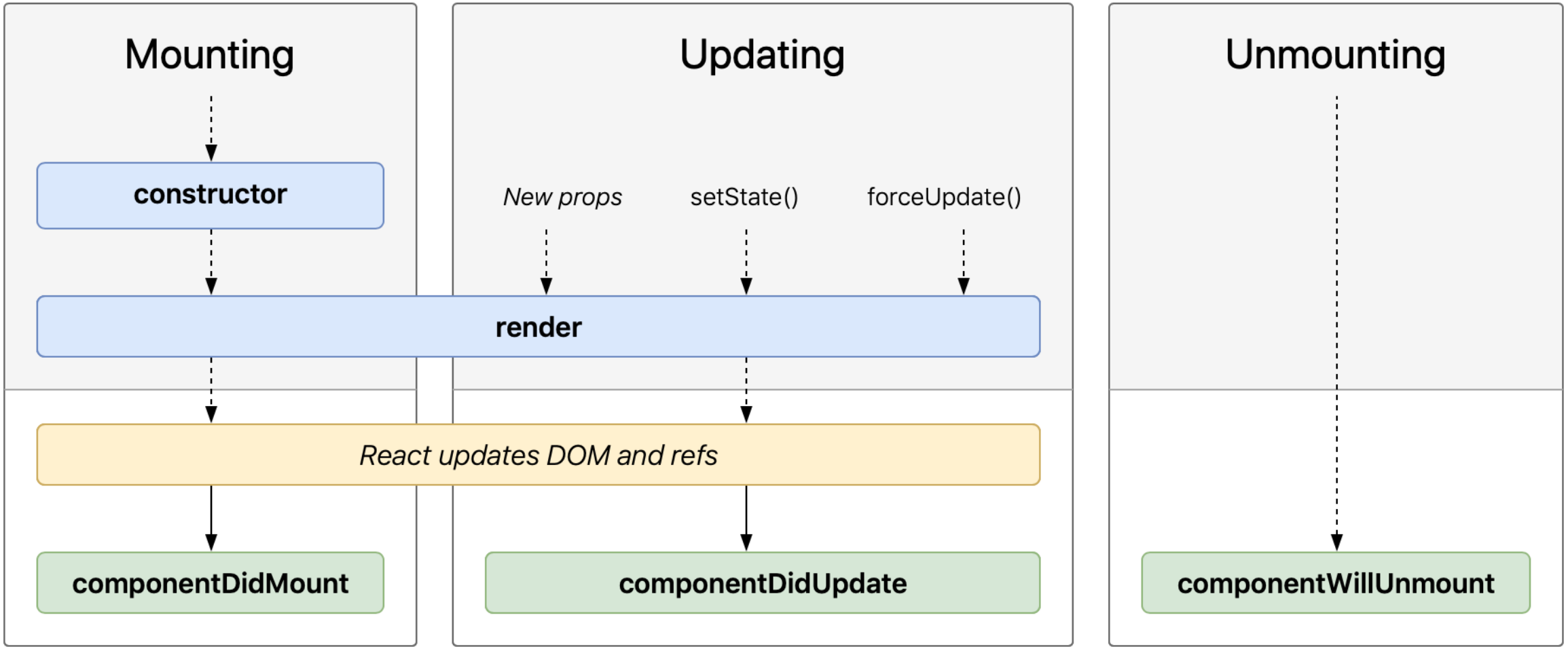

```
import './Greeting.css'

function Greeting({ firstName, lastName }: GreetingProps) {
return (
  <div className="greeting-card">
    <h1 className="greeting-title">Hello, {firstName} {lastName}!<h1>
  </div>
)}
```

React Components with State

React Hooks, Handling Events, State



React Component Lifecycle



React Hooks

SoftUni AI

- React **hooks** are functions, that provide **state** and **lifecycle features** inside functional components

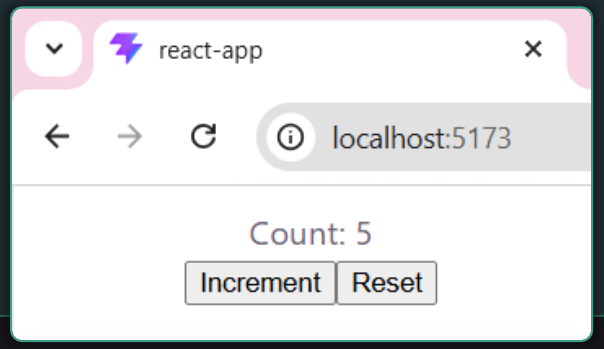
  - **useState()** → store data (local component state)
  - **useEffect()** → do something after render (e.g. load external data and re-render the component)
  - **useRef()** → remember something without re-render
  - **useMemo()** / **useCallback()** → optimize performance

The useState() Hook

SoftUni AI

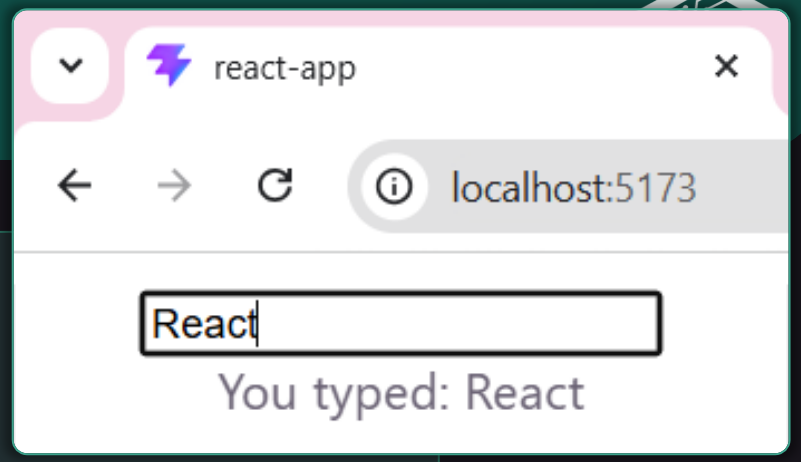
- Returns a **pair**
  - **[currentValue, setterFunc]**
  - Destructure to use both
- Assign an **initial value**
  - **useState(initial value)**
- Triggers **re-render**
  - Calling the **setter** function causes the component to **re-render** with the new value

```
import { useState } from "react";
export default function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count+1)}>
        Increment
      </button>
      <button onClick={() => setCount(0)}>
        Reset
      </button>
    </div>
  );
}
```



Handling Events in React

```
import { useState } from "react";

function InputExample() {
  const [value, setValue] = useState("");

  function handleChange(e: React.ChangeEvent<HTMLInputElement>) {
    setValue(e.target.value); // Change state → re-render
  }

  return (
    <div>
      <input value={value} onChange={handleChange} />
      <p>You typed: {value}</p>
    </div>
  );
}
```



Example: Sum Two Numbers

SoftUni AI

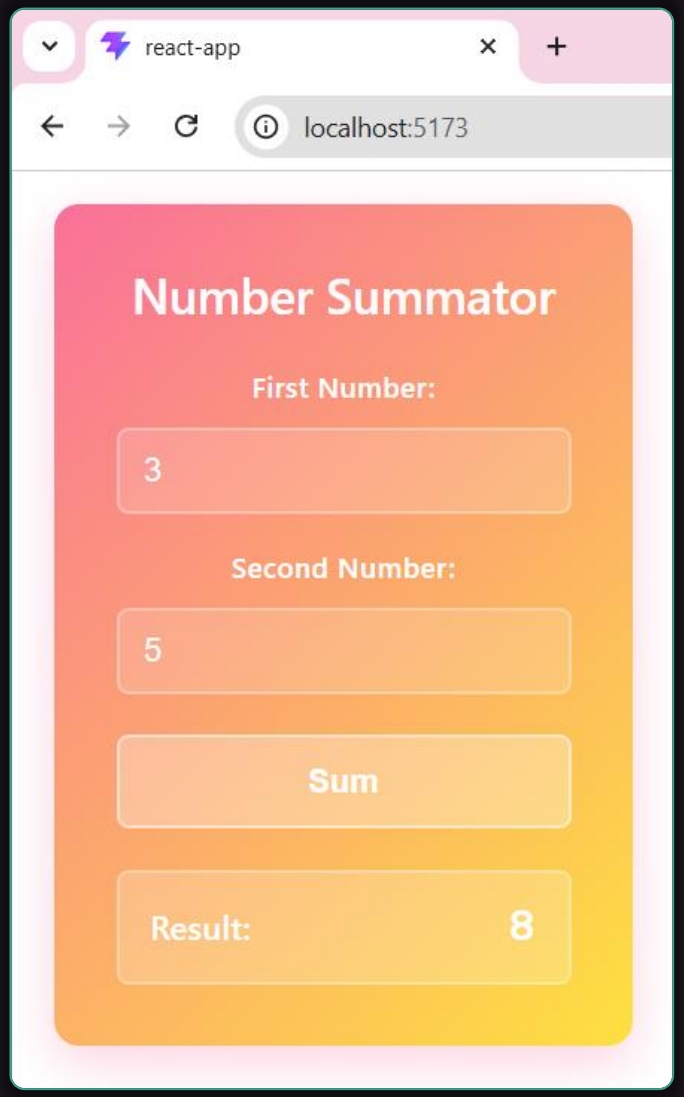
- AI prompt to create a React component **number summator**

Create a component **<NumberSummator />**: two **numbers** + **[Sum]** button + **result** field

NumberSummator.tsx

react-app > src > NumberSummator.tsx > ...

```
1  import { useState } from 'react'
2
3  export default function NumberSummator() {
4    const [firstNumber, setFirstNumber] = useState(0)
5    const [secondNumber, setSecondNumber] = useState(0)
6    const [result, setResult] = useState<number | null>(null)
7
8    const handleSum = () => {
9      setResult(firstNumber + secondNumber)
10   }
```



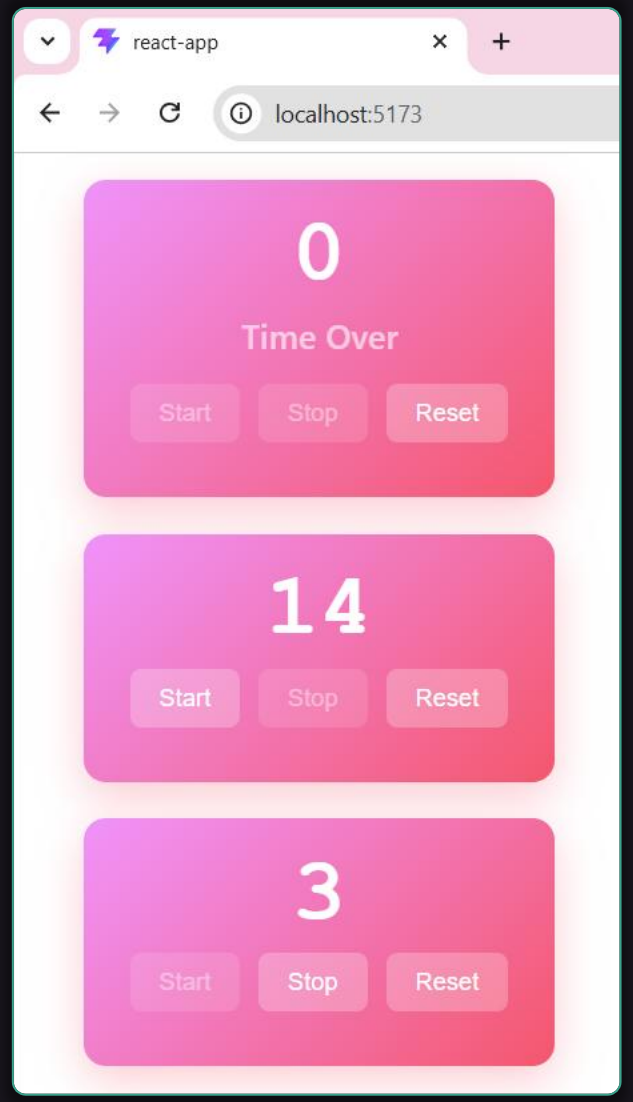
Example: Countdown Timer

SoftUni AI

- AI prompt to create a **countdown timer** React component

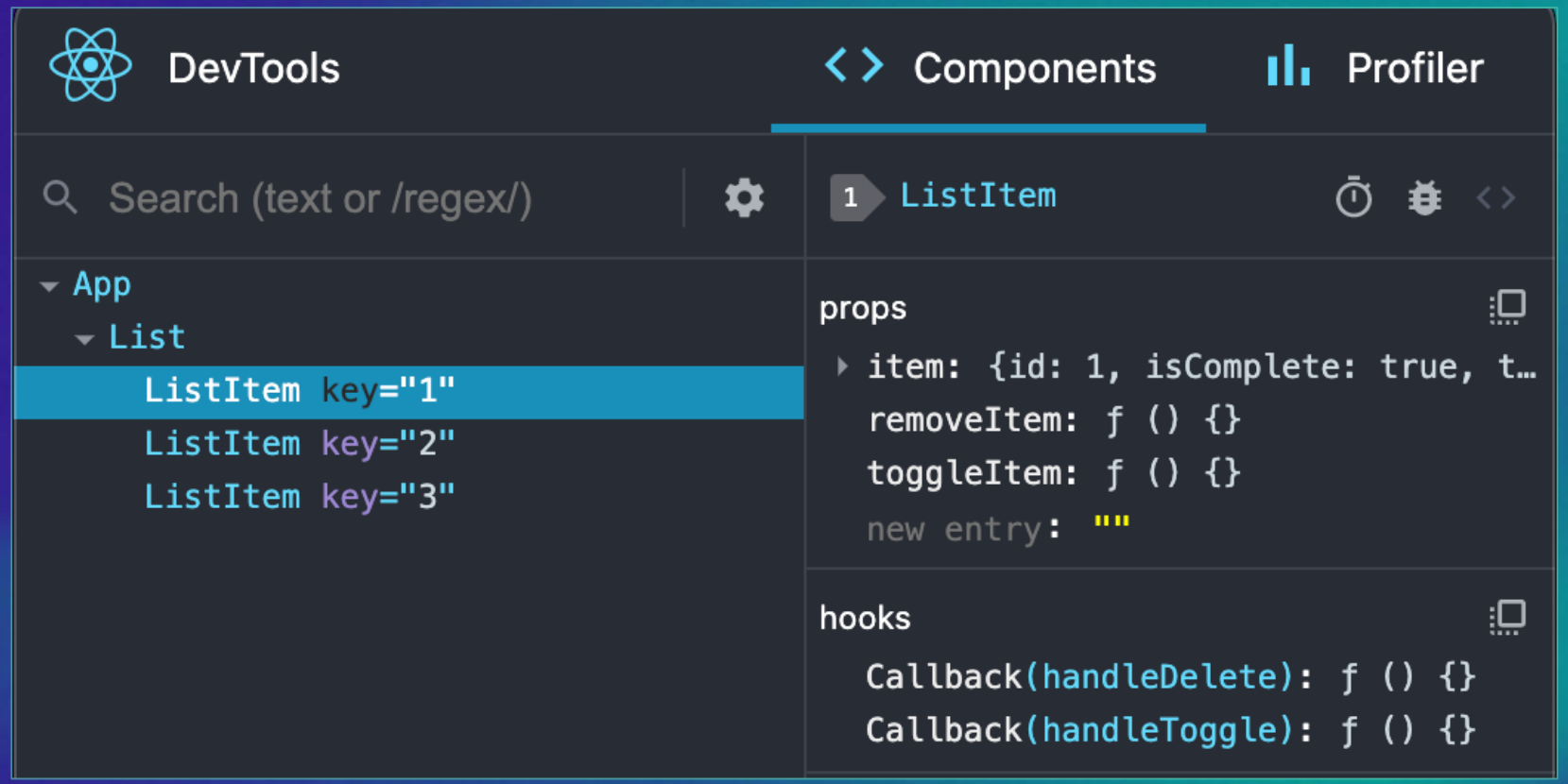
Create a component **CountdownTimer**
- Implement initial **value**, **start** button, **stop** button, **reset** button
- Stop the timer, when it reaches 0, and display blinking "**Time Over**"

Render **several <CountdownTimer />** components (use fragment)



React Dev Tools

Installing Dev Tools, Inspecting Components



React Dev Tools

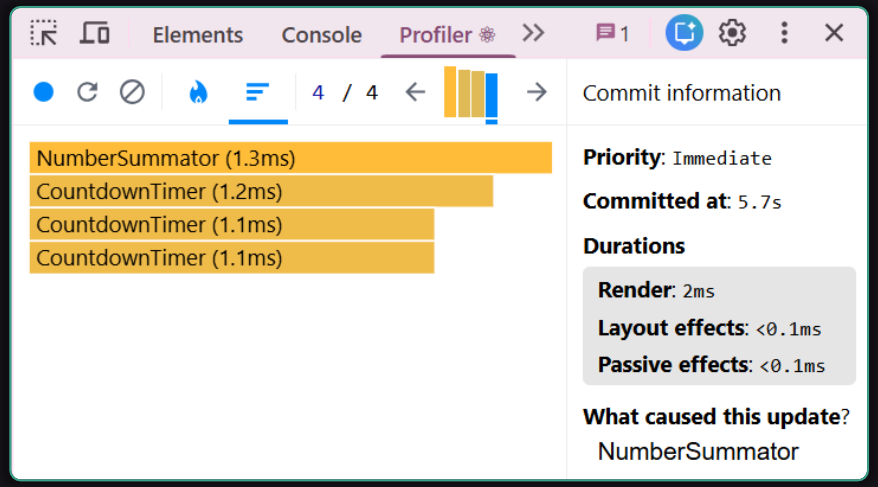
- **React Dev Tools** is a browser plugin for React developers
  - https://react.dev/learn/react-developer-tools

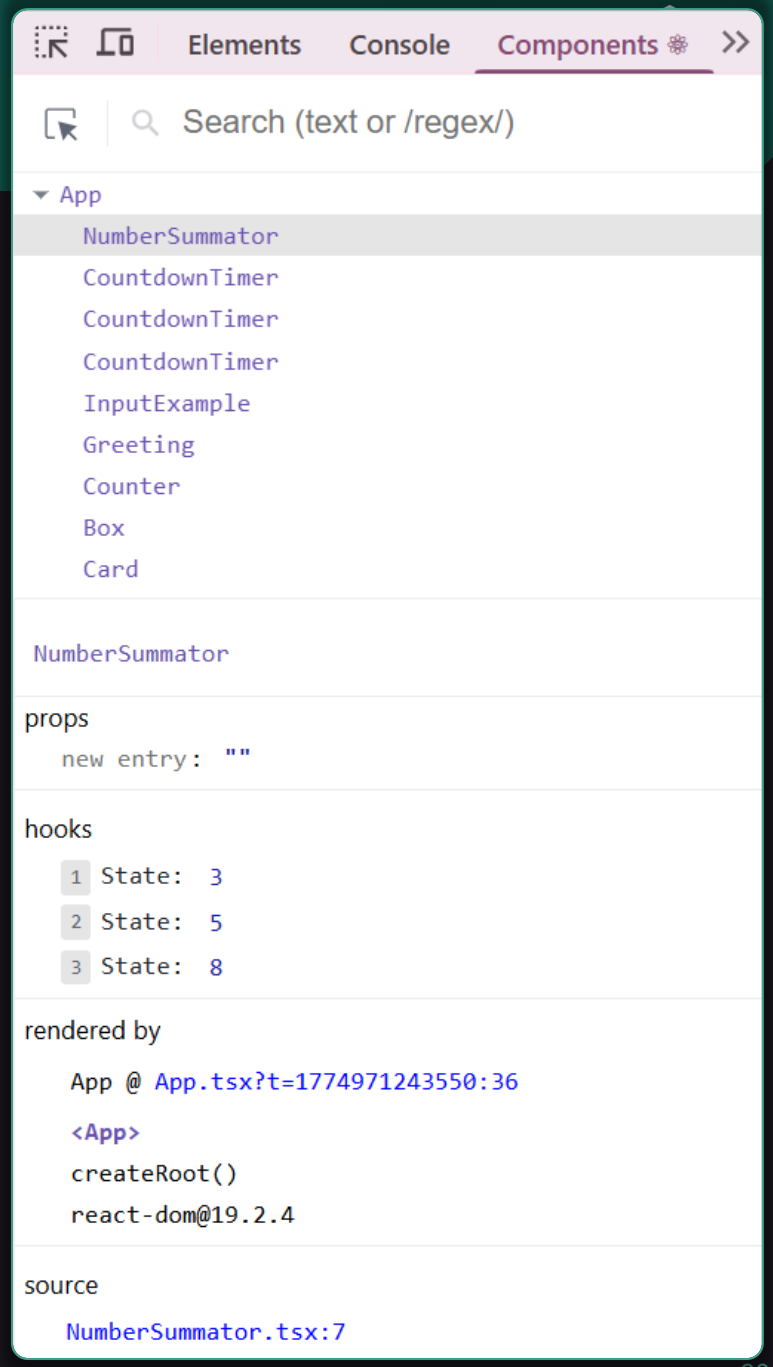
Components

Inspect component tree, props, and state, edit values live to test UI changes

Profiler

Measure rendering performance and identify bottlenecks





Inspecting App Components

SoftUni AI

react-app

localhost:5173

Number Summator

First Number:

5

Second Number:

7

Sum

Result: 12

Elements Console Sources Network Components 1

Search (text or /regex/)

App
Item
Area
NumberSummator
CountdownTimer
CountdownTimer
CountdownTimer
InputExample
Greeting
Counter
Box
Card

NumberSummator

props
new entry: ""

hooks
1 State: 5
2 State: 7
3 State: 12

rendered by
App @ App.tsx?t=1775039455212:25
<App>
createRoot()
react-dom@19.2.4

source
NumberSummator.tsx:7

Nested React Components

Components Holding Other Components

```
1   function InnerComponent()
2   {
3     return <div>...</div>;
4   }
5   function ParentComponent() {
6     return (
7       <div>
8         <InnerComponent />
9       </div>
```

Nested Components

- Components can render other components
  - **Component tree**: parent-child tree structure
  - Each component may hold **nested components**
- Component **composition**
  - Build complex UIs from simple, reusable parts
- Example of **component tree** in React

```
App
├─ Header
│   └─ NavBar
├─ Main
│   ├─ Sidebar
│   └─ Content
│       └─ ArticleList
│           └─ ArticleCard (×n)
└─ Footer
```

Example: Layout Component

```
interface LayoutProps {
  children: React.ReactNode
}

function Layout({ children }: LayoutProps) {
  return (
    <div className="layout">
      <Header />
      {children}
      <Footer />
    </div>
  )
}
```

```
function Header(){…}
```

```
function Footer(){…}
```

```
function App() {
  return (
    <Layout>
      <div>
        <Home />
      </div>
    </Layout>
  )
}
```

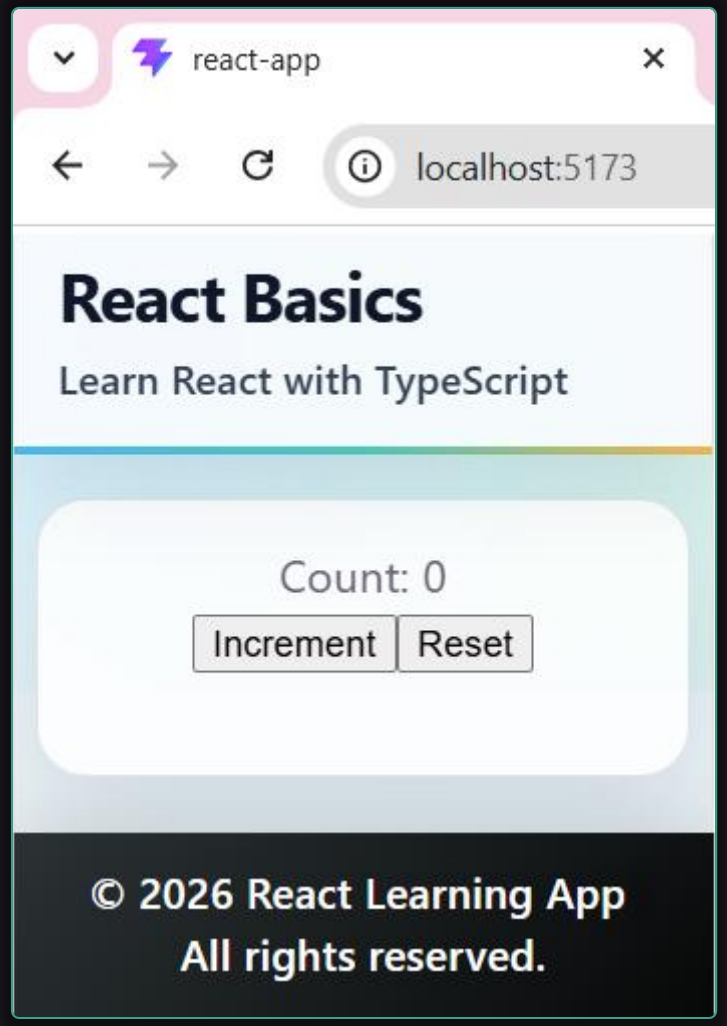
Layout with Header and Footer

SoftUni AI

- AI prompt to build a **layout component** holding **header** + page content + **footer**:

Define **layout components**:
- **<Layout><Header />...<Footer /></Layout>**
- **<Header />** (title + subtitle)
- **<Footer />** (copyright message)
- Define **CSS styles** for modern look and feel

Render the app content in the <Layout /> component



Layout with Header and Footer (2)

SoftUni AI

react-app

localhost:5173

React Basics

Learn React with TypeScript

Count: 0

Increment Reset

© 2026 React Learning App.
All rights reserved.

About Contact Privacy

Elements Components Network Application Console Sources

Search (text or /regex/)

App
Layout
Header
Counter
Footer

Counter

props
new entry: ""

hooks
1 State: 0

rendered by
App @ App.tsx?t=1775048550065:8
<App>
createRoot()
react-dom@19.2.4

source
Counter.tsx:7

Rendering Lists in React

Rendering Multiple Items, Conditional Rendering

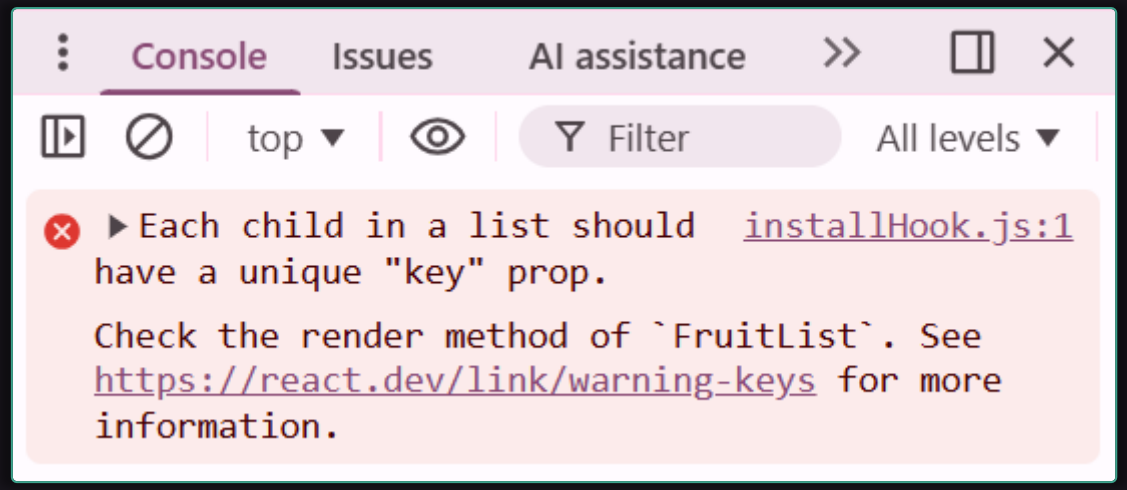
Rendering a Simple List

SoftUni AI

- Simple list rendering:

```
function FruitList() {
  const items = [
    "Apple", "Banana", "Orange"];
  return (
    <ul>
      {items.map(item =>
        <li>{item}</li>
      )}
    </ul>
  );
}
```

- This works for **static lists** (when items do not change / filter / reorder)
- React will show a **warning** about **missing key**



Rendering a List with a Key

SoftUni AI

- **Keys** uniquely identify list elements
  - Work with sorting, filtering, editing items
  - No warning messages

```
let contacts: Contact[] = [
  { id: 1, name: 'Steve' },
  { id: 2, name: 'Anna' },
  { id: 3, name: 'Boris' }
]
```

```
interface ContactListProps {
  contacts: Contact[]
}

function ContactList({ contacts }:
ContactListProps) {
  return (
    <div className="contacts-grid">
      contacts.map(c =>
        <ContactCard key={c.id} contact={c} />
    </div>
  )
}
```

Conditional Rendering

```
function ContactCard({ contact, isSelected }: ContactCardProps) {
  return (
    <div className={`card ${isSelected ? "selected" : ""}`}>
      <h3>{contact.name}</h3>
      <p>{contact.email}</p>
      {contact.phone && <p>{contact.phone}</p>}
      {isSelected
        ? <span className="badge">Selected</span>
        : <span className="badge">Click to select</span>
      }
    </div>
  )}
```

Ternary if-else

if (condition)
→ render a tag

Ternary if-else

Rendering a Contact List – Example

SoftUni AI

- AI prompt to create a **list of contacts** with filtering:

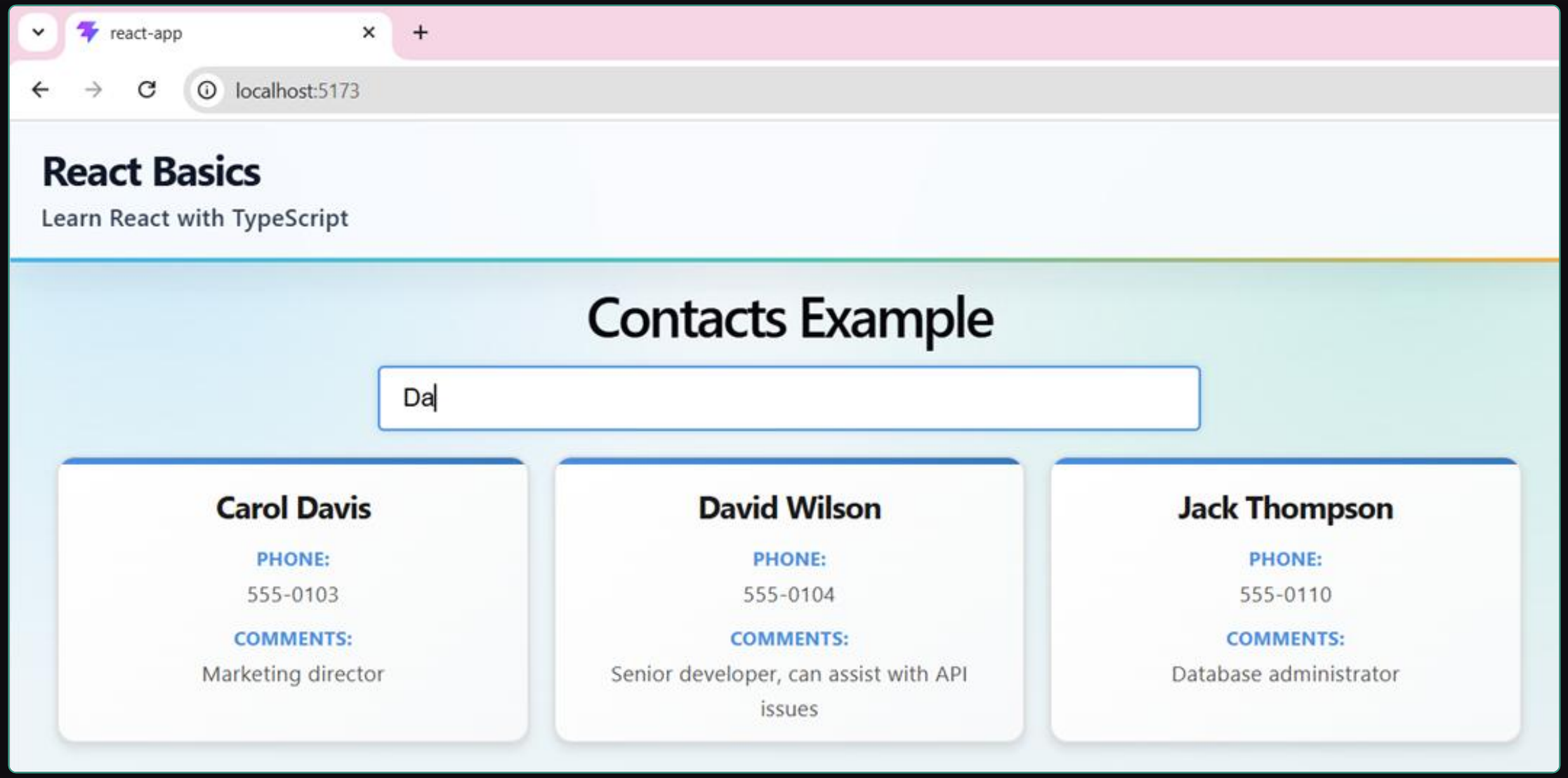
Create a **<ContactList />** component to render a list of contacts (name, phone, comments), with filtering (search by keyword)

- **Conditional rendering**: render "*No contacts found.*" for empty list
- Display the contacts as **cards** in a grid
- Use a child component **<ContactCard />**

Implement a component **<ContactsExample />**

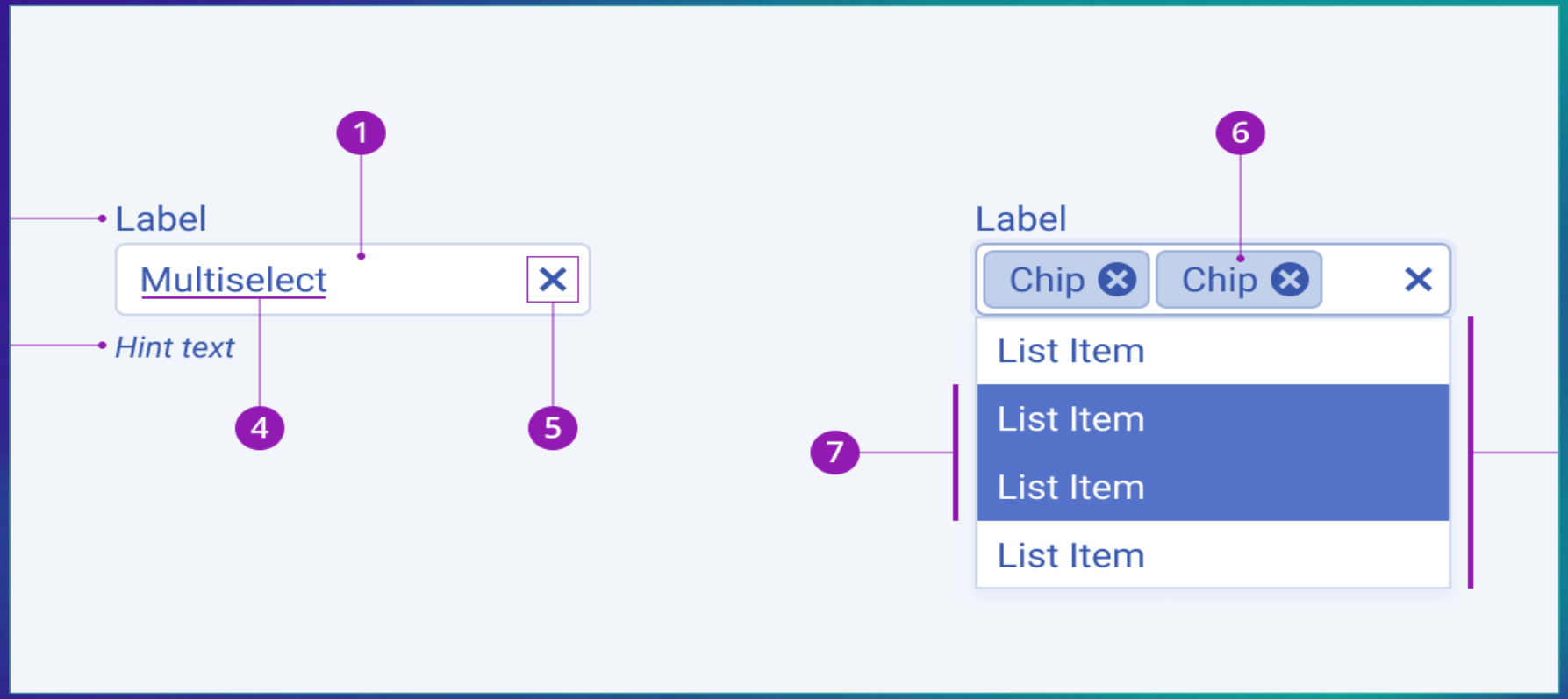
- Define a static set of 10-15 contacts
- Render the contacts using the <ContactList /> component

Rendering a Contact List – Result



Multi-Select Component

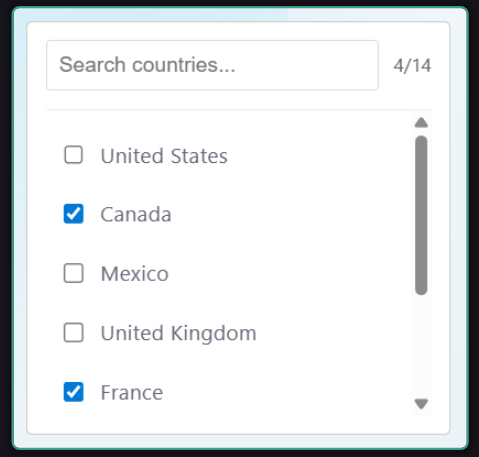
Creating a Reusable Component in React



Reusable Components in React

- In React, we often build **reusable components**
  - Child component take props + state → render its UI → expose event handlers → update parent state → re-render UI (parent + child)
  - Used later in the app, to build more complex the UI
- Example: we want to implement a **multi-select React component** (select multiple items from a list, with filtering)

```
<MultiSelect
  items={countries}
  selected={selectedCountries}
  onChange={setSelectedCountries}
  placeholder="Search countries …" />
```



Controlled Components

SoftUni AI

- In React, **controlled components** are a pattern for handling form inputs where **parent component owns the state**:
- **One-way data flow**
  - Parent **owns the state**
  - **Data** flows **down** (parent → child)
  - **Events** flow **up** (child → parent)

```
const countries = [
 { id: 'fr', label: 'France' },
 { id: 'cn', label: 'China' },
 { id: 'ca', label: 'Canada' },
 { id: 'es', label: 'Spain' }
];
```

```
const [selCountries, setSelCountries] = useState(["fr", "ca"]);
<MultiSelect items={countries} placeholder="Search countries …"
  selected={selCountries} onChange={setSelectedCountries} />
```

Example: Multi-Select Component

SoftUni AI

Create a minimalistic **multi-select React component**: <MultiSelect ... />

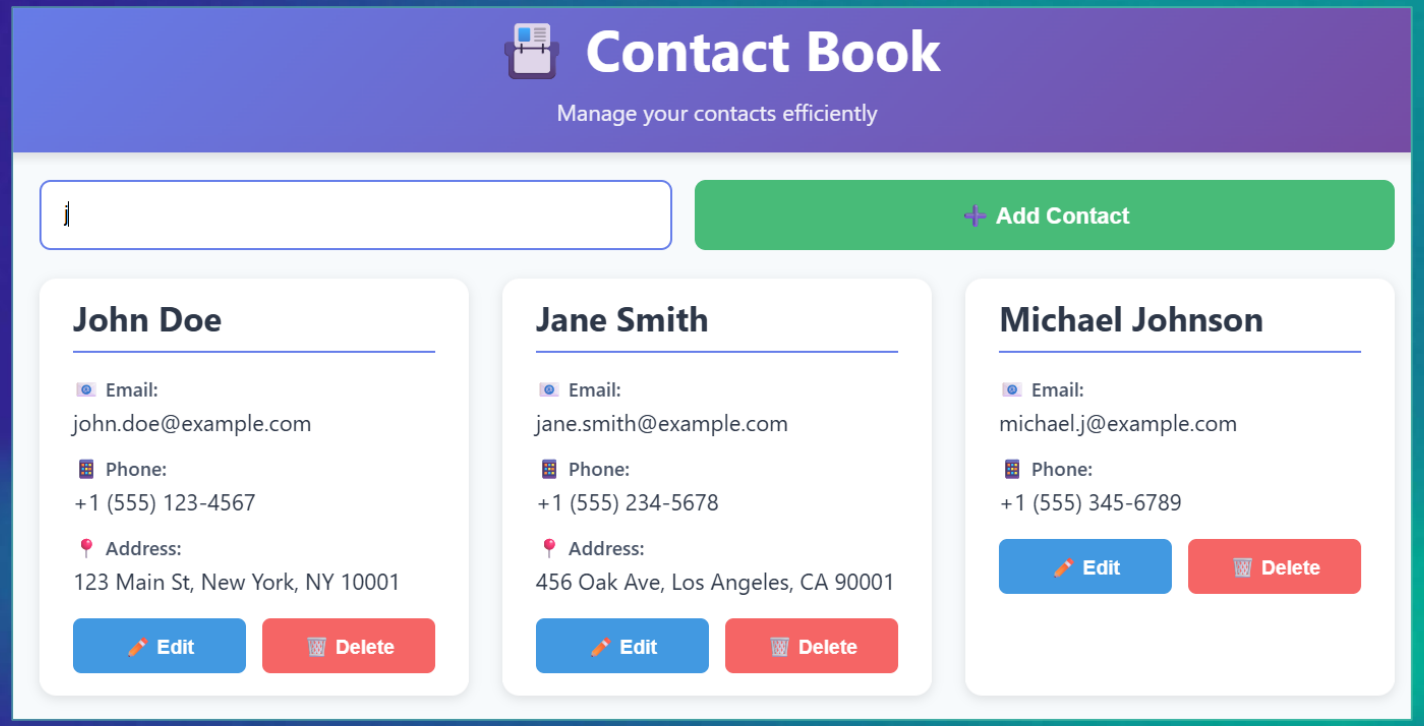
- Pass a list of items (id, label) and select a subset of the items
- Optionally, pass initially selected values
- Implement `onChange` event (when selected items change)
- Implement a search by substring for faster selection
- Render "*No options available*" when the items list is empty

Implement a component **<CountriesExample />**, which uses the multi-select component:

- Define a static set of 10-15 countries
- Define a state to hold selected countries
- Display selected contries with commas on selection change

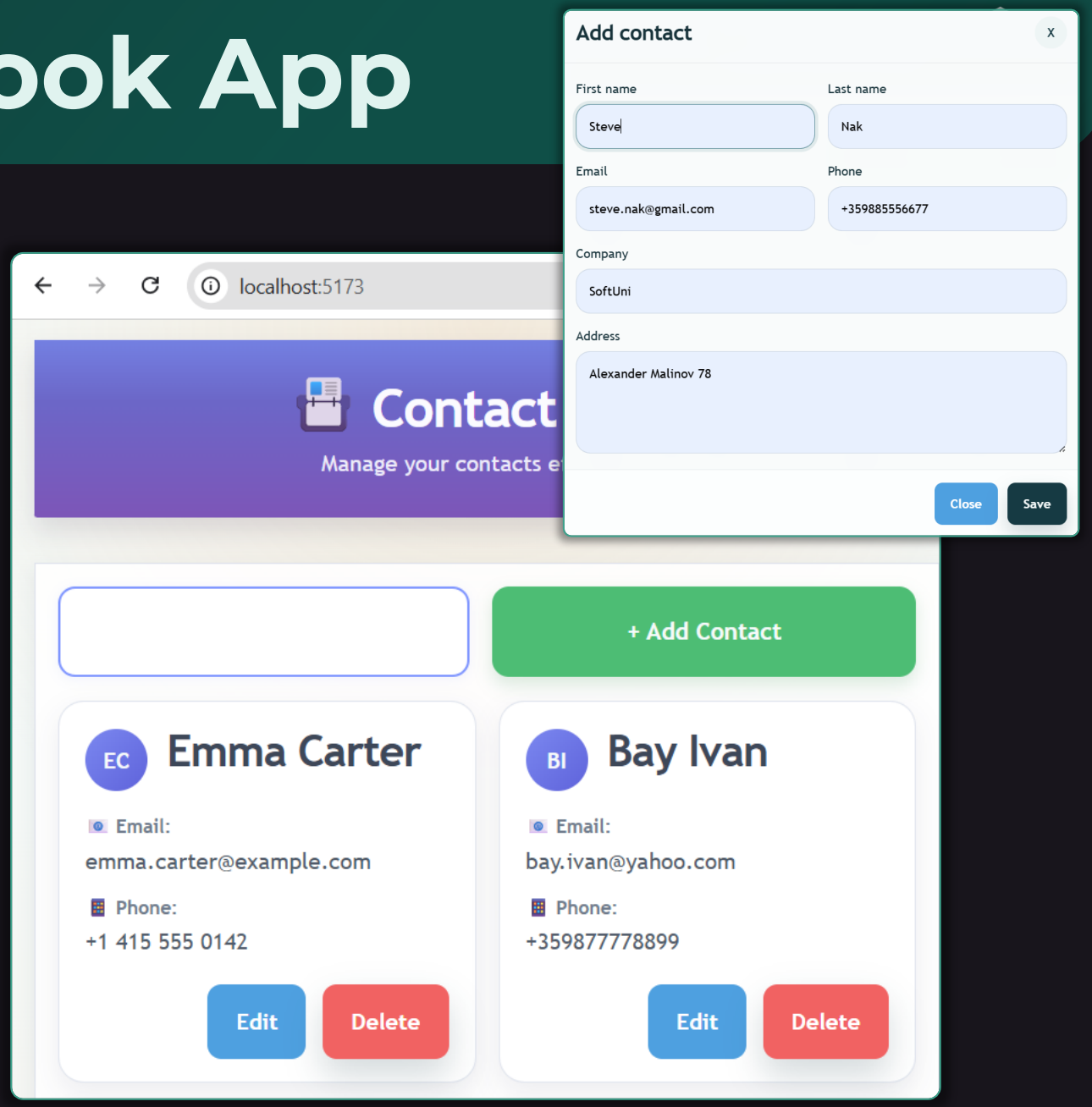
Exercise: Contactbook App

Creating a Contact Book App with React, TypeScript and Vite



Create a Contact Book App

- We want to build a React app to **manage contacts**
  - List / filter / add / edit / delete contacts
- Components:
  - **App → ContactsEditor**
  - **ContactCard**
  - **AddEditContactDialog, DeleteContactConfirmDialog**



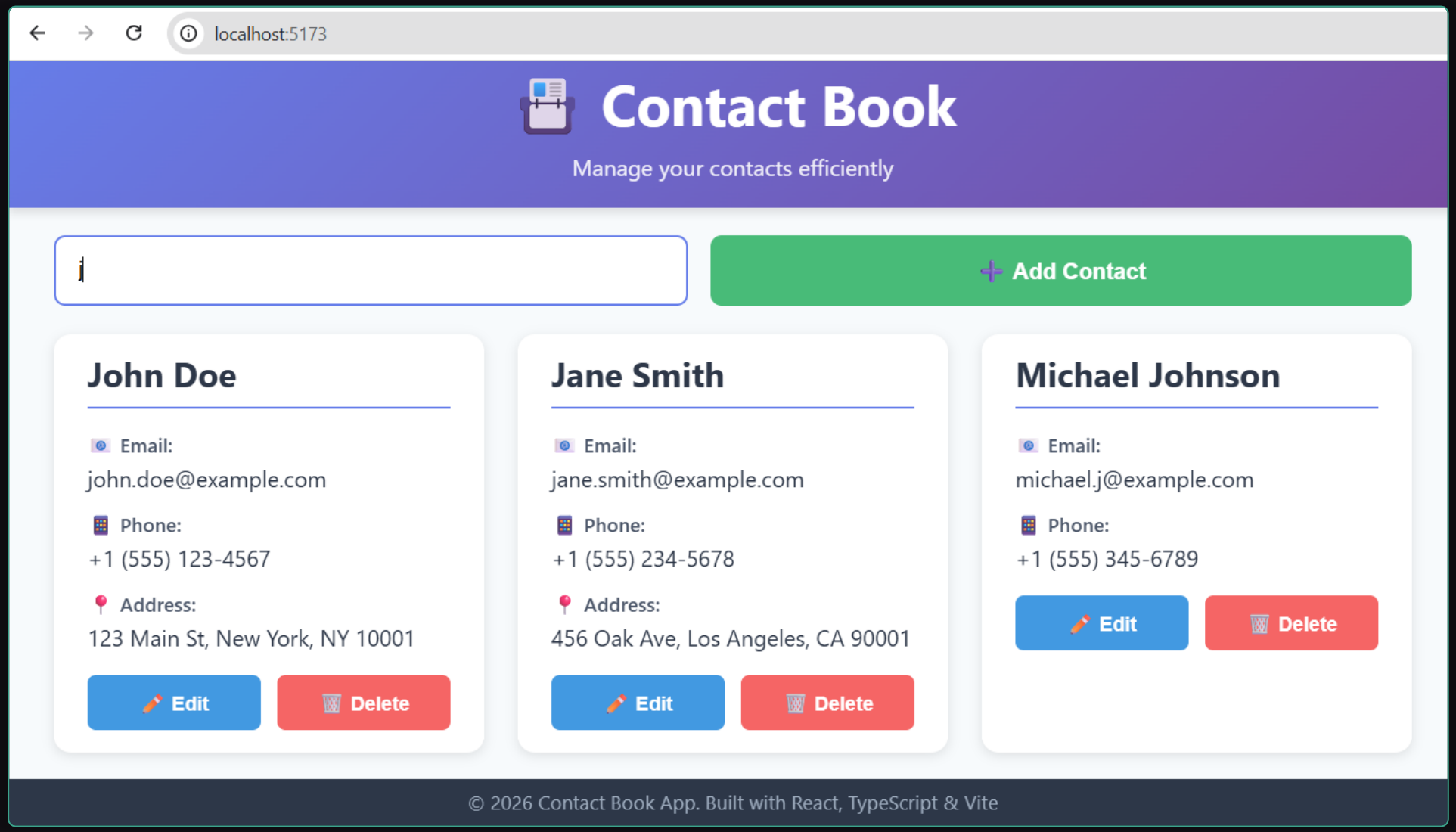
Contact Book App – AI Prompt

SoftUni AI

Create a **contact book app** with React, TypeScript and Vite
- Use layout with header + footer + app content
- Implement view contacts as cards (with a search filter)
- Implement add / edit / delete confirm with popups, as separate components
- Keep app data in the browser local storage
- Initially load a few sample contacts (for testing)

Split the app into the following React components:
- **App**: implements the app structure (header + contacts editor + footer)
- **ContactsEditor**: holds the contact data + implements list + filter / add / edit / delete
- **ContactCard**: view contact details, exposes onEdit, onDelete
- **AddEditContactDialog**: a modal dialog to add / edit contact, exposes onSave
- **DeleteContactConfirmDialog**: a modal dialog to confirm delete, exposes onConfirm

Contact Book App – Result



Contact Book App – Structure

- **App**: holds header + **ContactsEditor** + footer
- **ContactsEditor**: holds contacts data (state) + controls the render cards / filer / add / edit / remove contacts
  - **ContactCard**: contact data, onEdit + onDelete events
  - **AddEditContactDialog**: contact data, onSave event
  - **DeleteContactConfirmDialog**: contact data, onConfirm event

```
▾ App
  ▾ ContactsEditor
      ContactCard key="sample-1"
      ContactCard key="08d354f4-0ebf-4… "
      ContactCard key="sample-2"
      ContactCard key="be67d88f-0386-4… "
      ContactCard key="sample-3"
      ContactCard key="sample-4"
      AddEditContactDialog
      DeleteContactConfirmDialog
```

Lesson Summary

SoftUni AI

- **React** == component-based JavaScript UI library
- Build apps by composing UI **components**:

```
<App><Header/><News/><Footer/></App>
```

- **Props**: pass parameters to components

```
function MyButton({ label }) {
  return <button>{label}</button>;
}
```

- **State**: hold data in components

```
const [count, setCount] = useState(1);
<p>{count}</p>
<button onClick={() => setCount(0)}>Reset</button>
```

- **Data** flows parent → child, **events** flow child → parent

SoftUni AI

Questions?

© SoftUni AI – https://ai.softuni.bg. Copyrighted document. Unauthorized copy, reproduction or use is not permitted.

Postbank – Exclusive Partner for SoftUni AI

- One of the leading **banking institutions** in Bulgaria
- Member of the Eurobank Group with € 103 billion assets
- Innovative trendsetter with next generation **beyond banking**, transforming today, empowering tomorrow
- Certified **Top Employer 2025** by the international Top Employers Institute
- Proven people care and **wellbeing initiatives**
- Benefits and unlimited access to professional, **personal and leadership trainings and programs**
- www.postbank.bg / careers.postbank.bg

Diamond Partners of Software University

Software University

Coca-Cola HBC Bulgaria

Postbank

top EMPLOYER
България
Bulgaria
2025
FOR A BETTER WORLD OF WORK

SUPER HOSTING .BG

encorp.ai

createX

DRAFT KINGS
THE CROWN IS YOURS

INDEAVR
Serving the high achievers

Diamond Partners of SoftUni Digital

SoftUni Digital

Coca-Cola HBC Bulgaria

Postbank

top EMPLOYER
България
Bulgaria
2025
FOR A BETTER WORLD OF WORK

SUPER HOSTING .BG

encorp.ai

createX

DRAFT KINGS
THE CROWN IS YOURS

INDEAVR
Serving the high achievers

Diamond Partners of SoftUni Digital

SoftUni Digital

HUMAN

NETPEAK
DIGITAL GROWTH PARTNER

MarmaLAB | Е-комерс агенция

ETIENYANEV
Break Your Limits. Live Your Brand.

1 FOR FIT

leikov studio

Diamond Partners of SoftUni Creative

SoftUni Creative

Coca-Cola HBC Bulgaria

Postbank
top EMPLOYER
България
Bulgaria
2025
FOR A BETTER WORLD OF WORK

SUPER HOSTING .BG

encorp.ai

createX

DRAFT KINGS
THE CROWN IS YOURS

INDEAVR
Serving the high achievers

Organization Partners of SoftUni Creative

SoftUni Creative

creative corner

Studio π
Think Big • Design Smart

THE BUCKS TOWN'S WORK

DEFFECTO